# UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

## FACULTAD DE INGENIERÍA

---

## SISTEMAS DISTRIBUIDOS

---

# Laboratorio S05

*Mexi Banco con Docker Compose*


| | |
|---|---|
| **Nombre del alumno:** | Yukioayax Canek Gabriel Hernández |
| **Profesor:** | Oscar Manuel Ruiz Hurtado |
| **Grupo:** | 2 |
| **Semestre en curso:** | 2027-1 |


Ciudad Universitaria, CDMX

# Índice

**1. Introducción** **2**

**2. Objetivo** **2**

**3. Desarrollo** **2**
3.1. Paso 1 y 2: clonar y levantar con Compose . . . . . 2
3.2. Paso 3: la colección de Postman . . . . . 3
3.3. Paso 4: bajarlo con y sin volumen . . . . . 7
3.4. Análisis del Código Fuente (Paso 5) . . . . . 8
3.5. Estudio de Compose y Dockerfile (Paso 6) . . . . . 8

**4. Resultados** **10**

**5. Análisis de Resultados** **10**

**6. Conclusiones** **10**

# 1 Introducción

El presente reporte documenta el desarrollo del Laboratorio S05 enfocado en la orquestación de contenedores utilizando Docker Compose, el objetivo principal es desplegar una aplicación monolítica en Java (Mexi Banco) junto con su base de datos PostgreSQL, analizar el código fuente para comprender su estructura interna y probar la persistencia de datos mediante la manipulación de volúmenes de Docker.

# 2 Objetivo

Comprender el funcionamiento de `docker-compose.yml` para el levantamiento simultáneo de múltiples servicios, analizar la construcción de imágenes optimizadas a través de un `Dockerfile` de múltiples etapas (*multi-stage builds*) e inspeccionar la arquitectura de un monolito en Java basado en Spring Boot como paso previo a su separación en microservicios.

# 3 Desarrollo

## 3.1 Paso 1 y 2: clonar y levantar con Compose

Se clonó la etiqueta `v05.1` del repo (`git clone -branch v05.1 https://github.com/OscarRuiz21/mexi` y se levantó con `docker compose up -build -d`, primero se construye la imagen de `app`, después arranca `db`, compose espera a que diga *Healthy* y hasta entonces arranca `app`, con `docker compose ps` los dos salen **healthy** y no solo *running*, y el `/actuator/health` contesta `UP`.

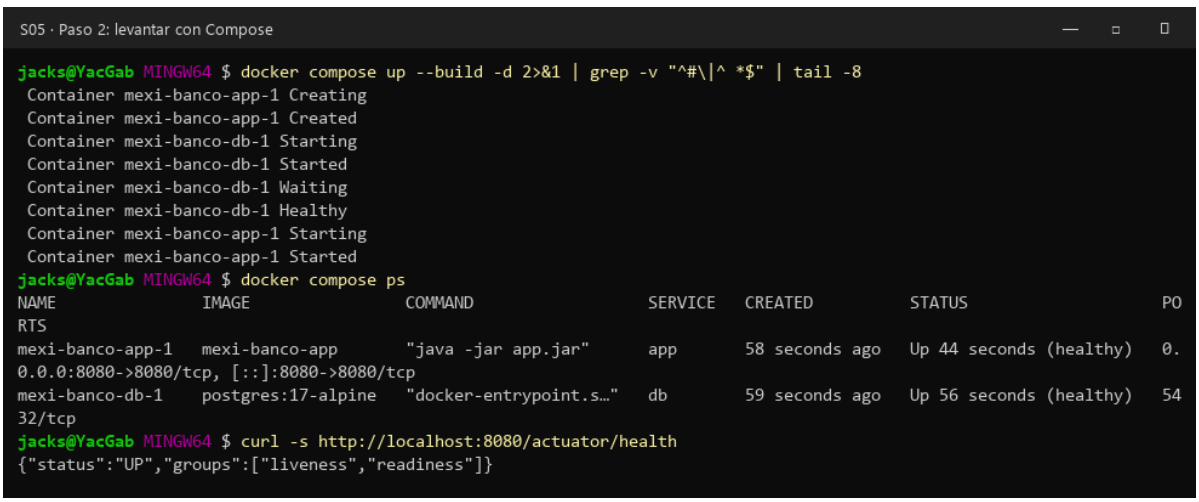



Figura 1: `docker compose up -build -d`, `docker compose ps` con los dos servicios *healthy* y la salud de la app.

## 3.2 Paso 3: la colección de Postman

La colección `mexi-banco-v05.1.postman_collection.json` se corrió completa y en orden con **newman**, que es el ejecutor de colecciones de Postman por línea de comandos, lo corrí desde su imagen de Docker y conectado a la red `mexibanco` que crea el compose, por eso la `baseUrl` es `http://app:8080` y no `localhost`, así cada petición manda lo mismo que con el botón *Send* y además se ven las pruebas que trae la colección. Las 20 peticiones respondieron lo esperado y las 34 verificaciones pasaron, sin ninguna falla.

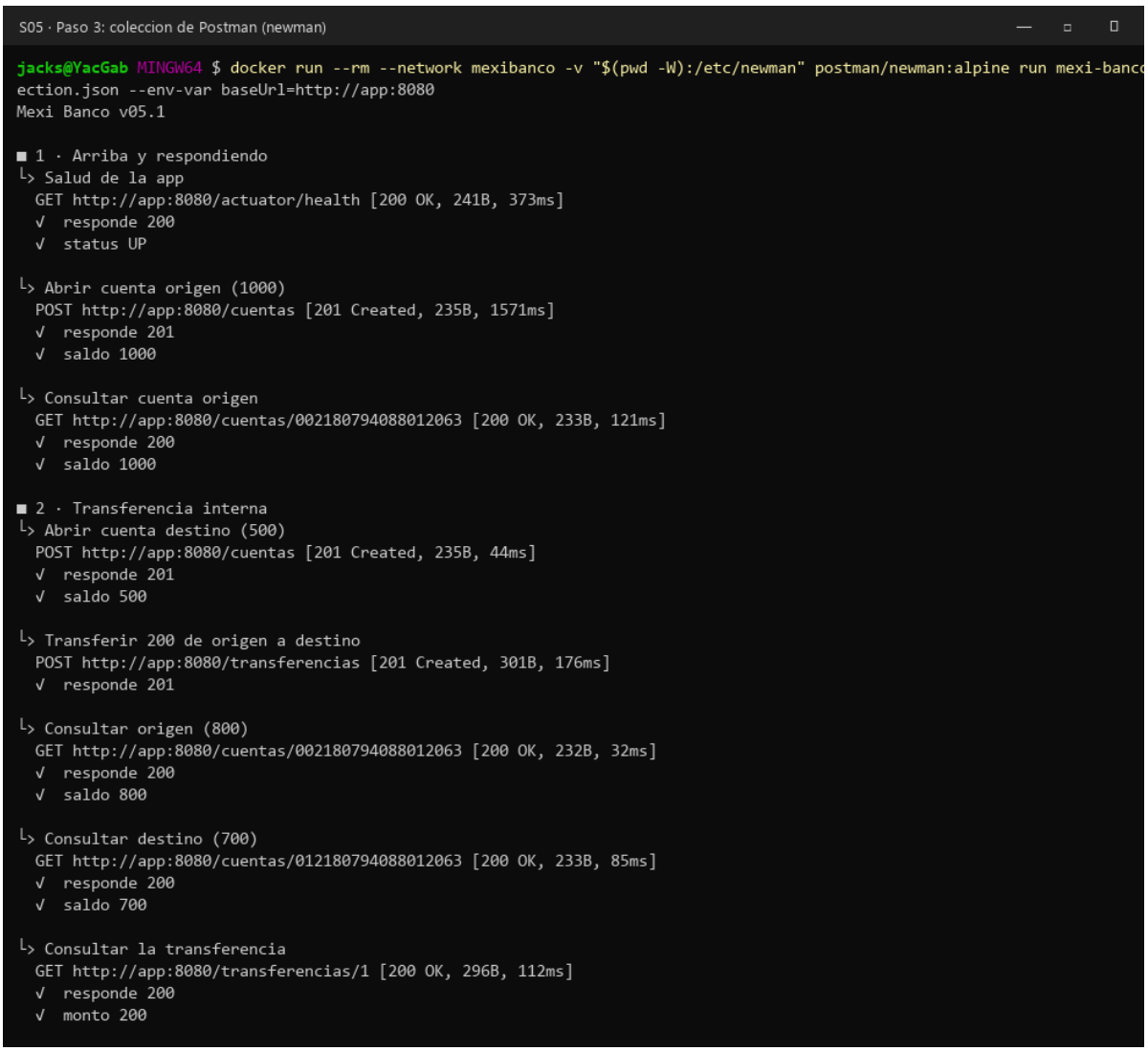



Figura 2: Grupos 1 y 2: salud, apertura de las cuentas de Ana (1000) y Beto (500), la transferencia de 200 y los saldos 800 y 700.

En el primer grupo se abre la cuenta de origen con 1000 y al consultarla regresa ese mismo saldo, en el segundo se abre la de destino con 500, se transfieren 200 y al consultar las dos cuentas quedan en **800** y **700**, y la transferencia se puede consultar por su id con su monto de 200.

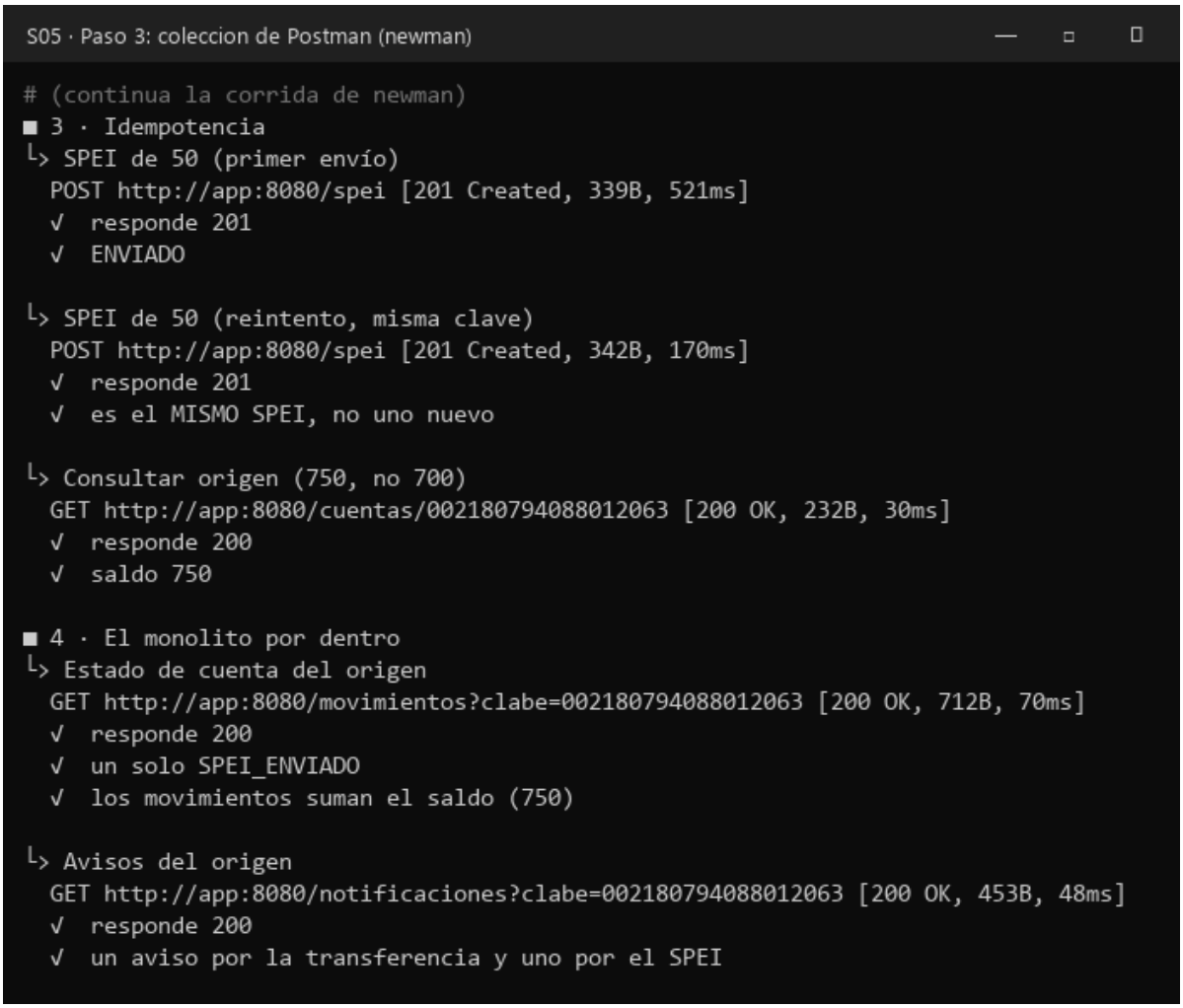



Figura 3: Grupos 3 y 4: el mismo SPEI mandado dos veces, el estado de cuenta y los avisos.

En el tercer grupo se manda el SPEI de 50 dos veces con la misma `Idempotency-Key`, las dos responden `201`, pero la segunda regresa **el mismo SPEI** y no uno nuevo, por eso el saldo queda en **750** y no en 700, el banco reconoce que es un reintento por la llave y no lo cobra otra vez. En el cuarto grupo el estado de cuenta tiene un solo `SPEI_ENVIADO` y los movimientos suman el saldo, y en los avisos hay uno por la transferencia y otro por el SPEI.

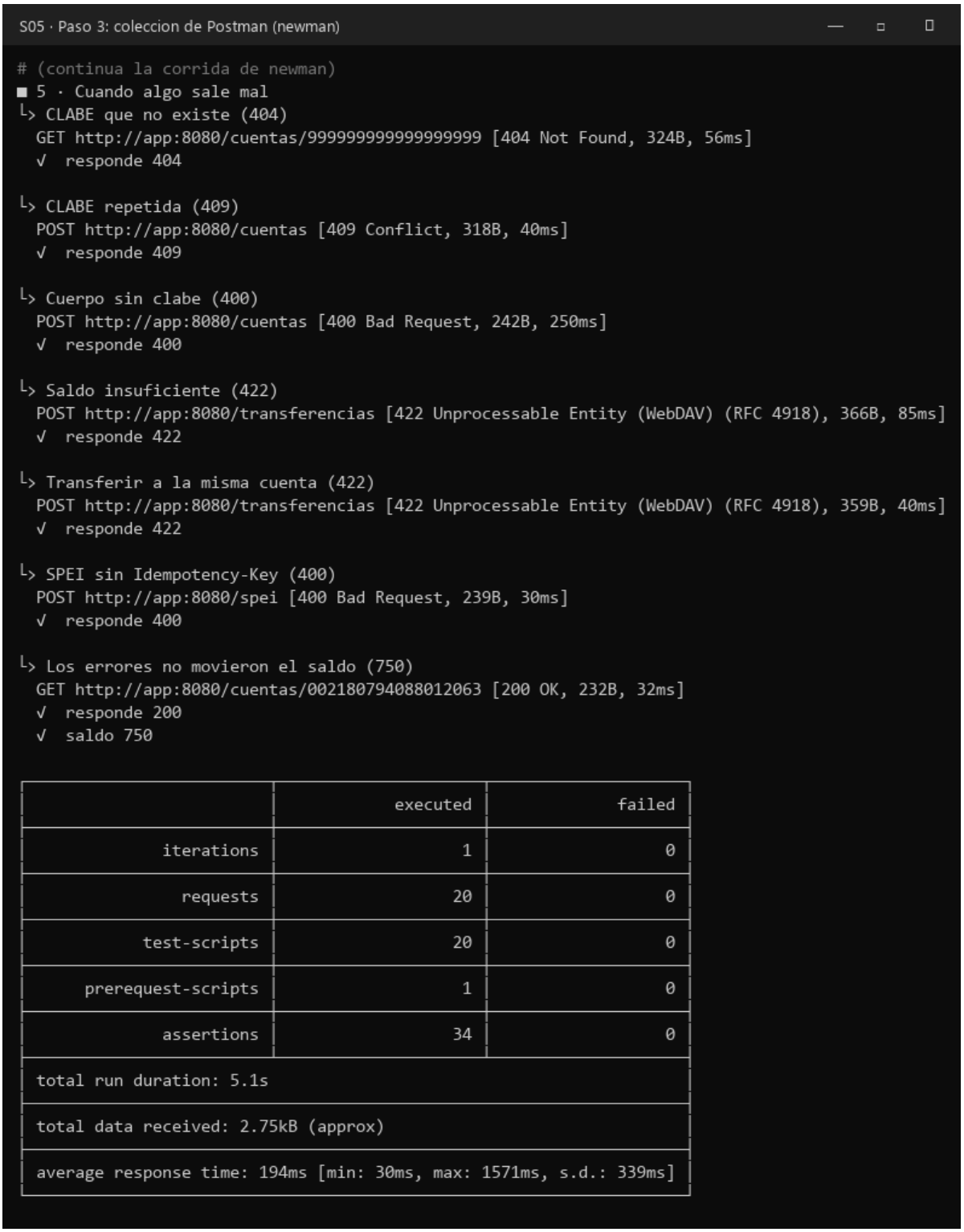



Figura 4: Grupo 5: los errores (404, 409, 400, 422, 422 y 400) y el resumen de la corrida, 20 peticiones y 34 verificaciones sin fallas.

En el quinto grupo se provocan los errores a propósito: una CLABE que no existe da **404**, abrir otra vez la misma CLABE da **409**, un cuerpo sin CLABE da **400**, transferir más de lo

que hay da **422** (saldo insuficiente), transferir a la misma cuenta también da **422** y un SPEI sin `Idempotency-Key` da **400**, y al final se consulta otra vez el origen y sigue en 750, o sea que ningún error movió dinero.
Para ver lo que contesta cada petición por dentro, repetí algunas con `curl`:

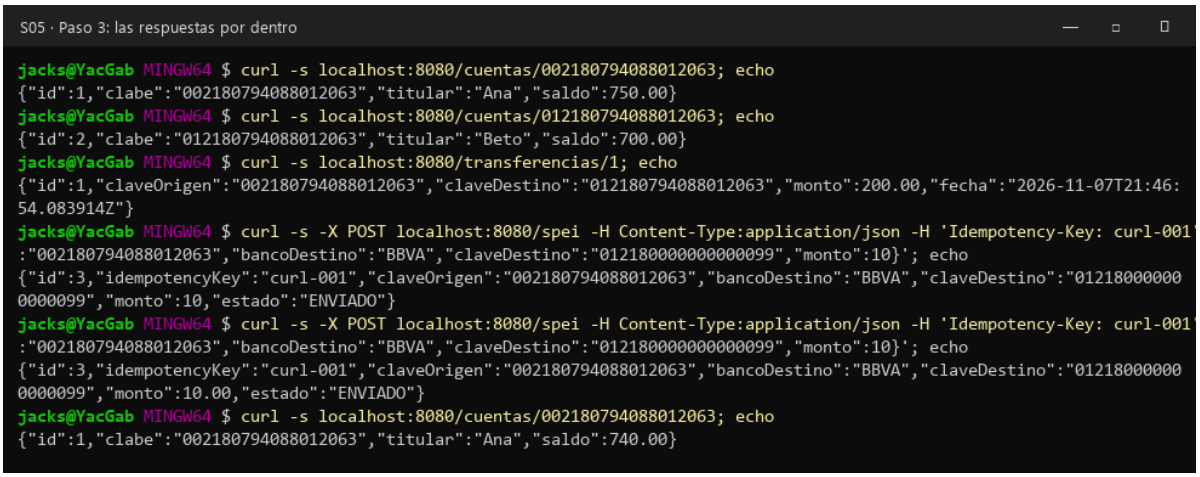



Figura 5: Cuentas de Ana (750) y Beto (700), la transferencia 1 y otro SPEI de 10 mandado dos veces con la llave `curl-001`.

Aquí se ve más claro lo de la idempotencia, las dos peticiones con la llave `curl-001` regresan el SPEI con `id 3`, y el saldo de Ana solo baja de 750 a 740, una sola vez.




Figura 6: Estado de cuenta y avisos de Ana, y el cuerpo de tres errores con su código HTTP.

En el estado de cuenta se ven los cuatro movimientos de Ana: el depósito inicial de 1000, la transferencia de $-200$, el SPEI de $-50$ y el de $-10$, con su saldo resultante en cada uno, y los avisos dicen lo mismo con texto. Los errores salen como *Problem Details* (`type`, `title`,

`status`, `detail`), el 404 dice `No existe la CLABE ...` y el 422 dice `Saldo insuficiente ...`, menos el 400 del SPEI sin llave, que sale con el formato de error por defecto de Spring porque lo rechaza antes de llegar al controlador.

## 3.3 Paso 4: bajarlo con y sin volumen

Primero lo bajé con `docker compose down`, se borraron los contenedores y la red, pero el volumen `mexi-banco_mexibanco_datos` siguió en `docker volume ls`, al levantarlo otra vez y consultar la cuenta de Ana, **seguía ahí** con su saldo de 740 y un 200, porque Postgres guarda sus archivos en ese volumen y el contenedor nuevo de `db` lo vuelve a montar con todo lo que tenía.

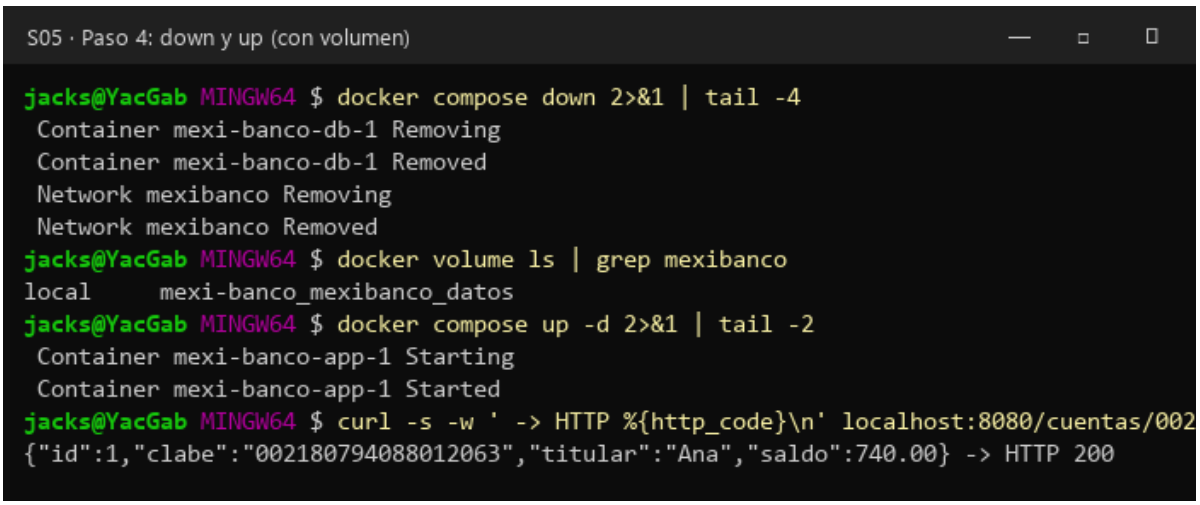



Figura 7: `down` y `up`: el volumen se queda y la cuenta de Ana sigue con 740.

Después lo bajé con `docker compose down -v`, ahora además se borró el volumen, y `docker volume ls` ya no lo encuentra, al levantar otra vez, Postgres arranca con un volumen nuevo y vacío, y la misma consulta da **404**, `No existe la CLABE`, porque esa base ya no tiene ninguna cuenta.

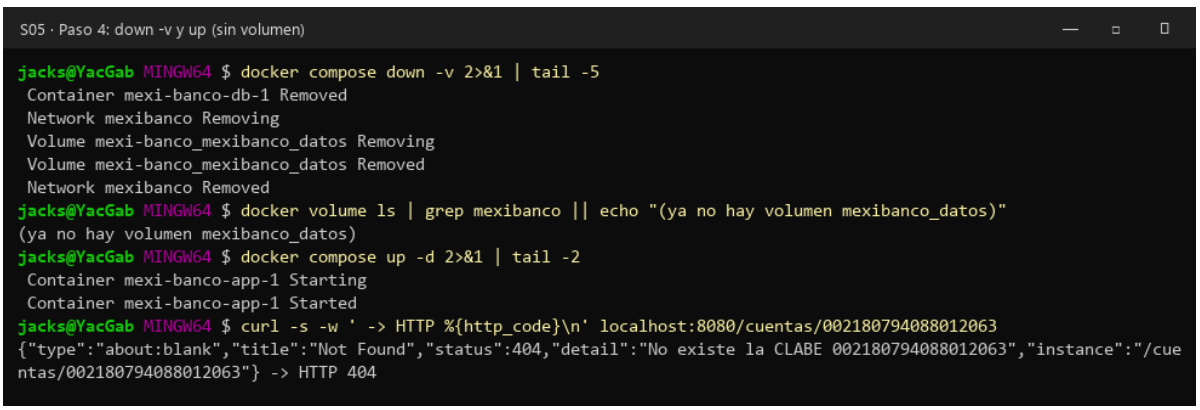



Figura 8: `down -v` y `up`: el volumen se borró y la misma consulta da 404.

O sea que lo que decide si los datos sobreviven no es el contenedor sino el volumen, el contenedor se puede tirar y volver a crear cuantas veces sea, mientras el volumen exista los datos siguen, y con `-v` se van junto con él.

## 3.4 Análisis del Código Fuente (Paso 5)

Al inspeccionar el proyecto Java, se identificó una arquitectura fuertemente modularizada, cada módulo (cuenta, movimiento, spei, transferencia) implementa un patrón estricto de capas, a continuación se muestran fragmentos clave que ilustran esta estructura:
**1. Capa de Controlador (Endpoints HTTP):** Expone las rutas REST e interactúa directamente con los servicios, sin contener reglas de negocio.

**CuentaController.java**

```
@RestController @RequestMapping(/cuentas") public class CuentaController
private final CuentaService cuentas;
@PostMapping @ResponseStatus(HttpStatus.CREATED) public Cuenta abrir(@Valid
@RequestBody AltaCuenta datos)  return cuentas.abrir(datos.clabe(),
datos.titular(), datos.saldoInicial());
```

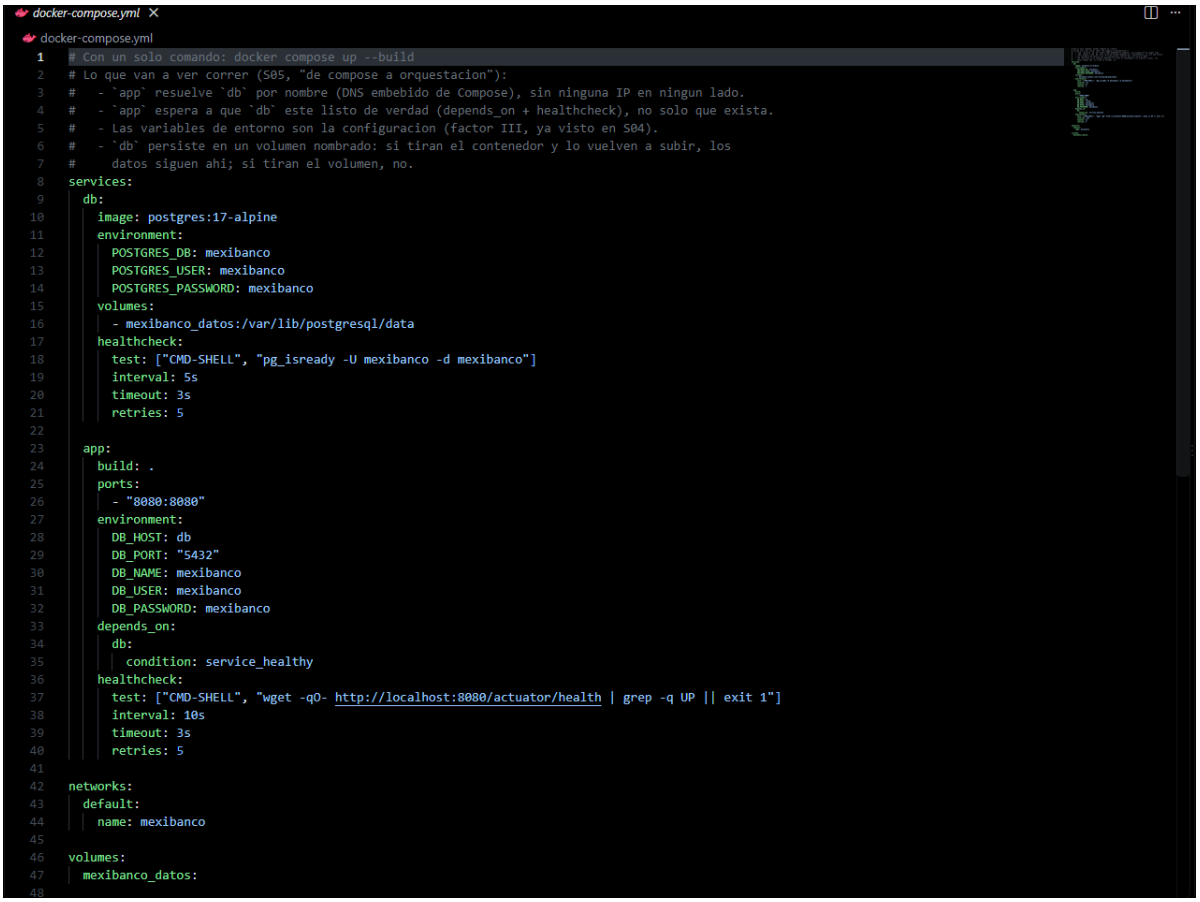
**2. Capa de Servicio (Lógica y Transacciones):** Centraliza las reglas del banco, en operaciones complejas, la anotación `@Transactional` garantiza la atomicidad (todo o nada), lo cual es posible porque actualmente todos los módulos comparten la misma base de datos.

**TransferenciaService.java**

```
@Service public class TransferenciaService  // ... @Transactional public
Transferencia transferir(String claveOrigen, String claveDestino,
BigDecimal monto)  // Cargo y abono atómico dentro del monolito
Cuenta origen = cuentas.cargar(claveOrigen, monto); Cuenta destino =
cuentas.abonar(claveDestino, monto);
Movimiento cargo = movimientos.registrar(origen.getClabe(), ...); Movimiento
abono = movimientos.registrar(destino.getClabe(), ...);
return transferencias.save(new Transferencia(claveOrigen, claveDestino,
monto));
```

## 3.5 Estudio de Compose y Dockerfile (Paso 6)

El archivo `docker-compose.yml` se encarga de la orquestación, su configuración más crítica es el `healthcheck` acoplado al `depends_on`.



```yaml
# Con un solo comando: docker compose up --build
# Lo que van a ver correr (S05, "de compose a orquestacion"):
#   - `app` resuelve `db` por nombre (DNS embebido de Compose), sin ninguna IP en ningun lado.
#   - `app` espera a que `db` este listo de verdad (depends_on + healthcheck), no solo que exista.
#   - Las variables de entorno son la configuracion (factor III, ya visto en S04).
#   - `db` persiste en un volumen nombrado: si tiran el contenedor y lo vuelven a subir, los
#     datos siguen ahi; si tiran el volumen, no.
services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: mexibanco
      POSTGRES_USER: mexibanco
      POSTGRES_PASSWORD: mexibanco
    volumes:
      - mexibanco_datos:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: mexibanco
      DB_USER: mexibanco
      DB_PASSWORD: mexibanco
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 5

networks:
  default:
    name: mexibanco

volumes:
  mexibanco_datos:
```

Figura 9: Archivo docker-compose.yml mostrando healthchecks y dependencias.

El servicio `app` no arranca simplemente porque `db` exista, sino que espera a que la condición `service_healthy` se cumpla, asegurando que PostgreSQL esté listo para recibir peticiones. Por otro lado, la construcción de la imagen se define en el `Dockerfile` utilizando dos etapas (*multi-stage build*).



```dockerfile
# ---- ETAPA 1: CONSTRUCCION ----
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY mvnw .
COPY .mvn .mvn
RUN ./mvnw dependency:go-offline -B
COPY src ./src
RUN ./mvnw clean package -DskipTests

# ---- ETAPA 2: EJECUCION ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Figura 10: Archivo Dockerfile en dos etapas.

La primera etapa (`builder`) utiliza el JDK completo para descargar dependencias y compilar el `.jar`. La segunda etapa usa únicamente el JRE, copiando el binario compilado.

# 4 Resultados

Se logró desplegar exitosamente la aplicación Mexi Banco y su base de datos de manera coordinada. La colección de Postman corrió completa con newman, con 20 peticiones y 34 verificaciones sin ninguna falla, desde abrir cuentas y transferir hasta los errores, y el SPEI repetido con la misma llave no se cobró dos veces. Asimismo, se comprobó experimentalmente la diferencia crítica entre bajar los contenedores preservando el volumen y destruyéndolo (`-v`), con `down` la cuenta de Ana siguió ahí y con `down -v` la misma consulta dio 404.

# 5 Análisis de Resultados

La práctica evidenció la importancia de condicionar el inicio de los contenedores; sin la directiva `service_healthy` en el archivo Compose, la aplicación de Spring Boot habría fallado al intentar conectarse a un PostgreSQL que aún estaba en proceso de inicialización, en cuanto al `Dockerfile`, el uso de múltiples etapas demostró ser una técnica esencial: aislar el entorno de compilación (JDK) del de ejecución (JRE) no solo reduce significativamente el tamaño final de la imagen, sino que disminuye su superficie de ataque al no incluir código fuente ni herramientas de desarrollo en producción.
Finalmente, el análisis del código Java reveló los retos futuros: operaciones como `transferir`, que actualmente se aseguran con una simple etiqueta `@Transactional` en el monolito, requerirán soluciones de consistencia eventual cuando las cuentas y los movimientos se separen en contenedores distintos.

# 6 Conclusiones

Docker Compose simplifica drásticamente el despliegue de entornos multicontenedor, permitiendo definir redes, volúmenes y dependencias lógicas en un solo archivo declarativo, comprendí que el ciclo de vida de los datos está directamente ligado a la gestión de los volúmenes, siendo imperativo manejar comandos como `down -v` con extrema precaución en entornos productivos, por último, la inspección del código monolítico sirvió para sentar las bases teóricas de por qué migrar a microservicios exige un rediseño profundo en la comunicación y las transacciones de las aplicaciones distribuidas.